TODO Tom

Okostelefon és IoT eszközök programozása feladat

Készítette: Martinák Mátyás

2026. április 26.

Tartalomjegyzék

1. Bevezetés **2**

2. Rendszeráttekintés **2**
2.1. Technológiai stack . . . 2
2.2. Magas szintű működés . . . 2

3. Backend struktúra felépítése **2**
3.1. Build és konfigurációs állományok . . . 2
3.2. Alkalmazás indítópont . . . 2
3.3. Domain modell és API szerződés . . . 2
3.4. REST végpontok és hibakezelés . . . 3
3.5. Üzleti logika és perzisztencia . . . 3
3.6. Időzített működés és séma migráció . . . 3

4. Frontend felépítése **3**
4.1. Android projekt konfigurálás . . . 3
4.2. UI és klienlogika . . . 3
4.3. Emlékeztető infrastruktúra . . . 3

5. Frontend használat – teljes UI bemutatás **3**
5.1. Fő képernyő elemei . . . 3
5.2. Task létrehozása – kattintási folyamat . . . 4
5.3. Task módosítása – kattintási folyamat . . . 7
5.4. Task késszerre jelölése / visszaállítása . . . 8
5.5. Task törlése . . . 10

6. Tesztelés **10**

7. Összefoglalás **11**

1. Bevezetés

A TODO Tom egy Kotlin alapú, kétrétegű alkalmazás: egy Spring Boot backendből és egy Android (Jetpack Compose) frontendből áll. A rendszer célja klasszikus TODO funkcionalitás biztosításán túl olyan haladó funkciók kezelése, mint a prioritási szintek, határidők, emlékeztetők, automatikus kategóriába sorolás és ismétlődő feladatok.

A dokumentáció célja, hogy a rendszer architektúráját, fő állományait és működési logikáját részletesen bemutassa.

2. Rendszeráttekintés

2.1. Technológiai stack

- Backend: Kotlin, Spring Boot, Spring Web, Spring Data JPA, SQLite
- Frontend: Android, Kotlin, Jetpack Compose, Retrofit, WorkManager
- Kommunikáció: REST API (JSON)
- Adattárolás: lokális SQLite adatbázis (`taskmanager.db`)

2.2. Magas szintű működés

1. A felhasználó a mobil kliensben létrehoz/módosít egy feladatot.
2. A frontend Retrofit segítségével HTTP kérést küld a backendnek.
3. A backend Controller – Service – Repository feldolgozza a kérést.
4. A JPA/SQLite réteg perzisztálja a változást.
5. A frontend frissíti az UI állapotot, szükség esetén emlékeztetőt ütemez.

3. Backend struktúra felépítése

3.1. Build és konfigurációs állományok

A backend build-folyamata és függőségei a `build.gradle.kts` állományban vannak definiálva. A projektnév a `settings.gradle.kts`-ben, míg a futási paraméterek az `application.yml`-ben találhatók.

3.2. Alkalmazás indítópont

A `TaskManagerApplication.kt` az alkalmazás bootstrap osztálya. Itt történik a Spring kontextus inicializálása és az időzített folyamatok engedélyezése.

3.3. Domain modell és API szerződés

A `Task.kt` tartalmazza az entitásmodell definícióját, beleértve az enumerátorokat (prioritás, kategória, ismétlődés). A `TaskDtos.kt` a klienssel cserélt request/response modelleket írja le.

3.4. REST végpontok és hibakezelés

A `TaskController.kt` adja a CRUD végpontokat. A hibakezelés két komponensre oszlik: `TaskNotFoundException.kt` (domain kivétel) és `TaskErrorHandler.kt` (egységes API hibaválasz).

3.5. Üzleti logika és perzisztencia

A `TaskService.kt` kezeli a tényleges üzleti szabályokat: normalizálás, auto-kategorizálás, állapotváltás. A `TaskRepository.kt` biztosítja az adatelést JPA-n keresztül.

3.6. Időzített működés és séma migráció

A `TaskRecurrenceScheduler.kt` időzített módon generálja az ismétlődő feladatok következő példányait. A `TaskSchemaMigration.kt` gondoskodik róla, hogy frissítéskor a szükséges oszlopok létrejöjjenek az SQLite táblában.

4. Frontend felépítése

4.1. Android projekt konfigurálás

A frontend külön Android projektben van. A plugin- és modulbeállításokat az alábbi állományok tartalmazzák:

4.2. UI és klienlogika

A `MainActivity.kt` egyesíti a klienst: API modellek, Retrofit interfész, ViewModel, Compose UI komponensek, valamint a létrehozás/szerkesztés/törlés/státusz váltási folyamatokat.

4.3. Emlékeztető infrastruktúra

A `ReminderWorker.kt` WorkManager alapú background worker, amely rendszertesítéseket küld az időzített emlékeztetőkről.

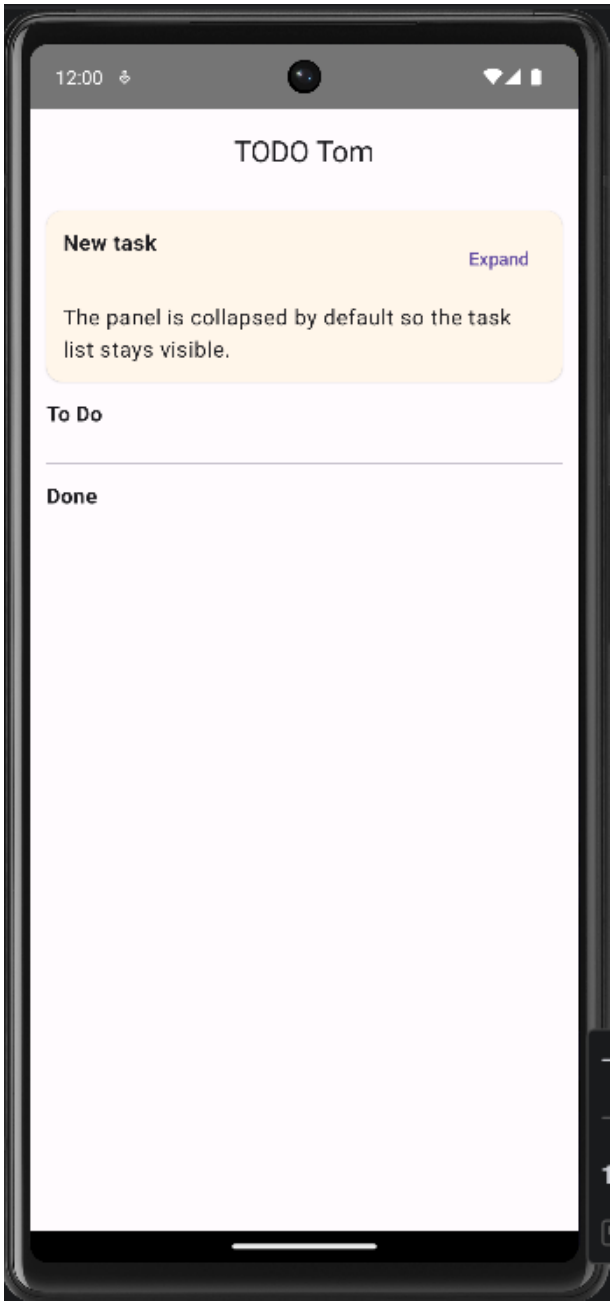
5. Frontend használat – teljes UI bemutatás

5.1. Fő képernyő elemei

A mobil képernyő fő elemei:

- Top bar: alkalmazáscím (TODO Tom)
- New task panel (Expand/Collapse)
- Input mezők: Title, Description
- Választó elemek: Priority, Category, Recurrence
- Dátum/idő elemek: Due date, Reminder

- Action gomb: Create task
- Két lista: To Do és Done
- Task action chipek: Mark done / Move back to TODO, Edit, Delete



1. ábra. A TODO Tom alkalmazás fő UI felülete

5.2. Task létrehozása – kattintási folyamat

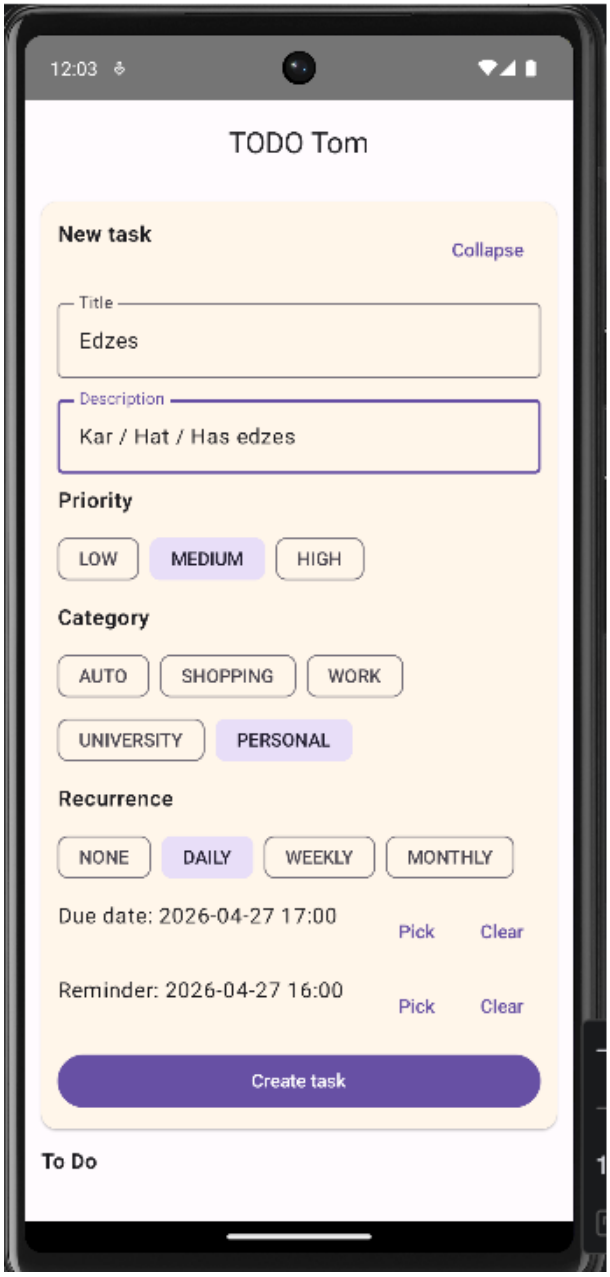
1. A `New task` panelnél az `Expand` gombra kattintva lehet új Task-ot lérehozni.
2. A `Title` mező kitöltése kötelező, ez lesz az új feladat azonosítója.

3. Opcionálisan a `Description` tartalmát feltöltve további leírást adhatunk az új Taskról.

4. A `Priority`, `Category`, `Recurrence` értékek beállításával további kifejtést adhatunk meg a feladatról. Kategorizálhatjuk, megadhatjuk, hogy hányszor ismétlődik.

5. A felhasználó választhat `Due date` és `Reminder` időpontot. Emlékeztető kiválasztásakor a felhasználó értesítést kap a telefonjára, ha a Task aktuálissá válik.

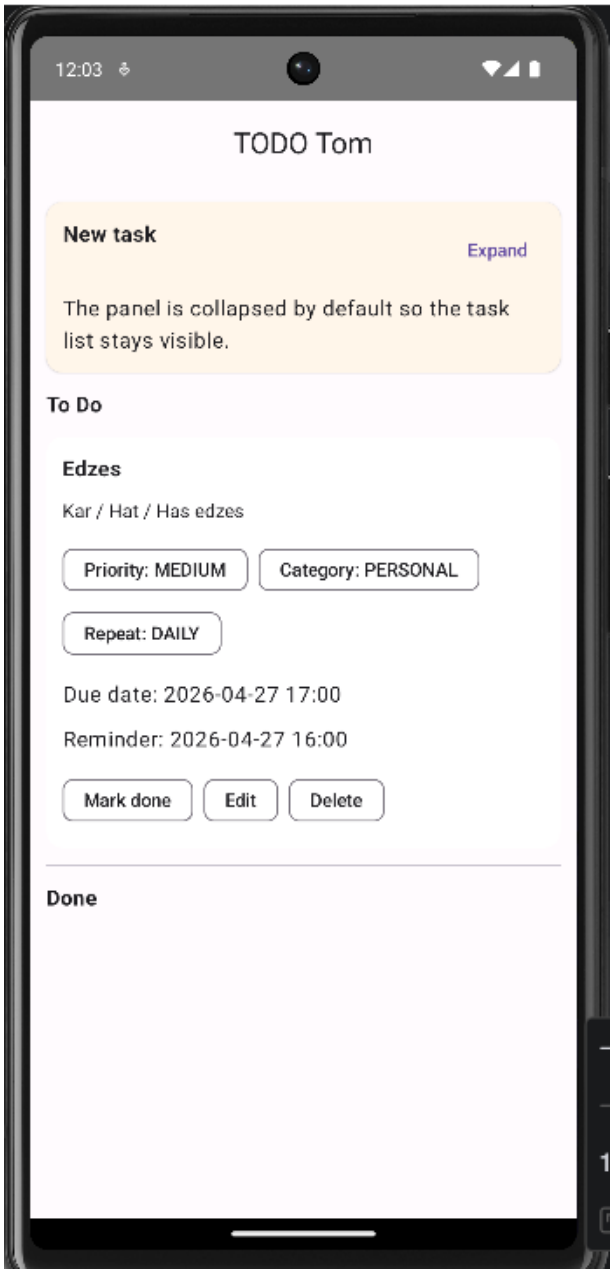
6. A `Create task` gombra kattintva eltárolódik az SQLITE adatbázisban az új Task entry.



2. ábra. Task létrehozása a TODO Tom-ban

```
[
  {
    "id": "54777cbb-a898-4cd4-933a-98f674bedf52",
    "title": "Edzes",
    "description": "Kar / Hat / Has edzes",
    "priority": "MEDIUM",
    "deadlineAt": "2026-04-27T17:00:00Z",
    "reminderAt": "2026-04-27T16:00:00Z",
    "category": "PERSONAL",
    "recurrence": "DAILY",
    "completed": false,
    "completedAt": null,
    "sourceTaskId": null,
    "createdAt": "2026-04-26T12:03:50.237Z",
    "updatedAt": "2026-04-26T12:03:50.237Z"
  }
]
```

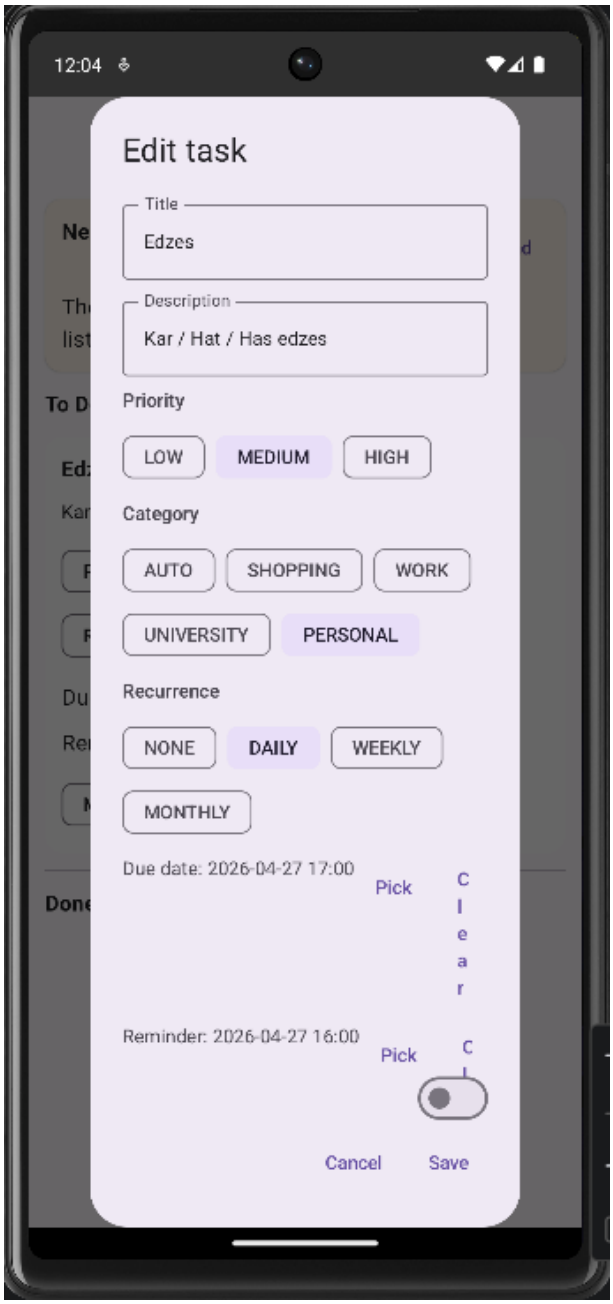

3. ábra. Az új Task entryje az adatbázisban



4. ábra. Feladatok kilistázása az alkalmazásban

5.3. Task módosítása – kattintási folyamat

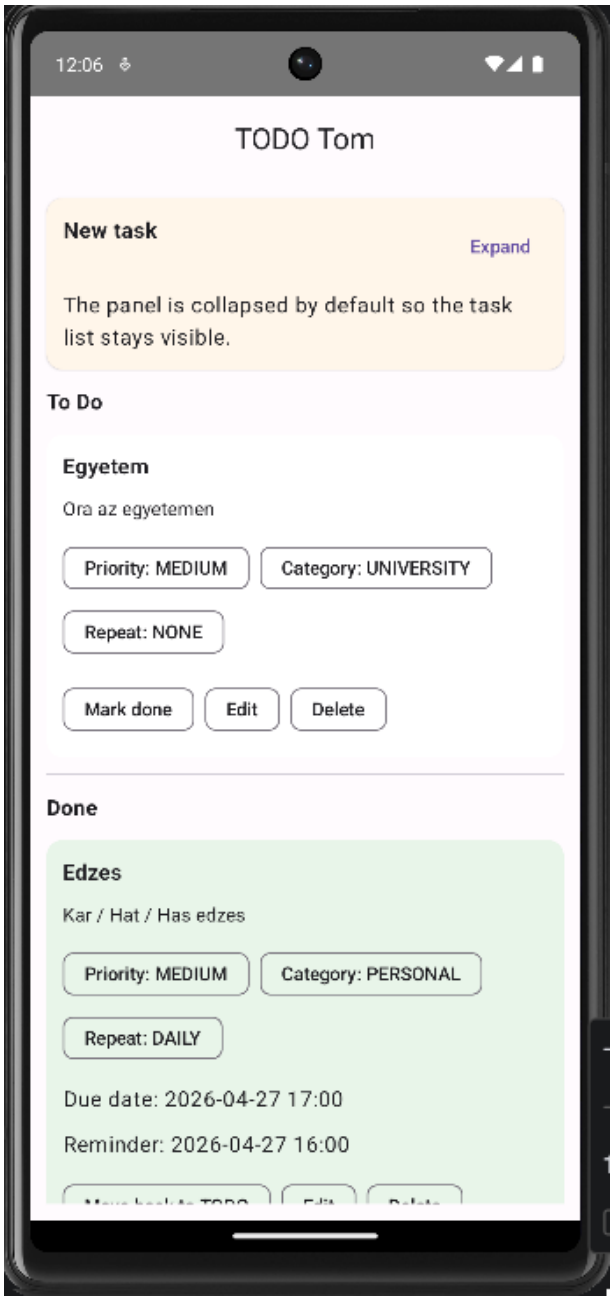
1. Az `Edit`-re történő kattintás során a felhasználó szerkesztheti a Taskot.
2. Az `Edit task` párbeszédben módosíthatóak a feladat eredeti paraméterei.



5. ábra. A Task szerkesztése

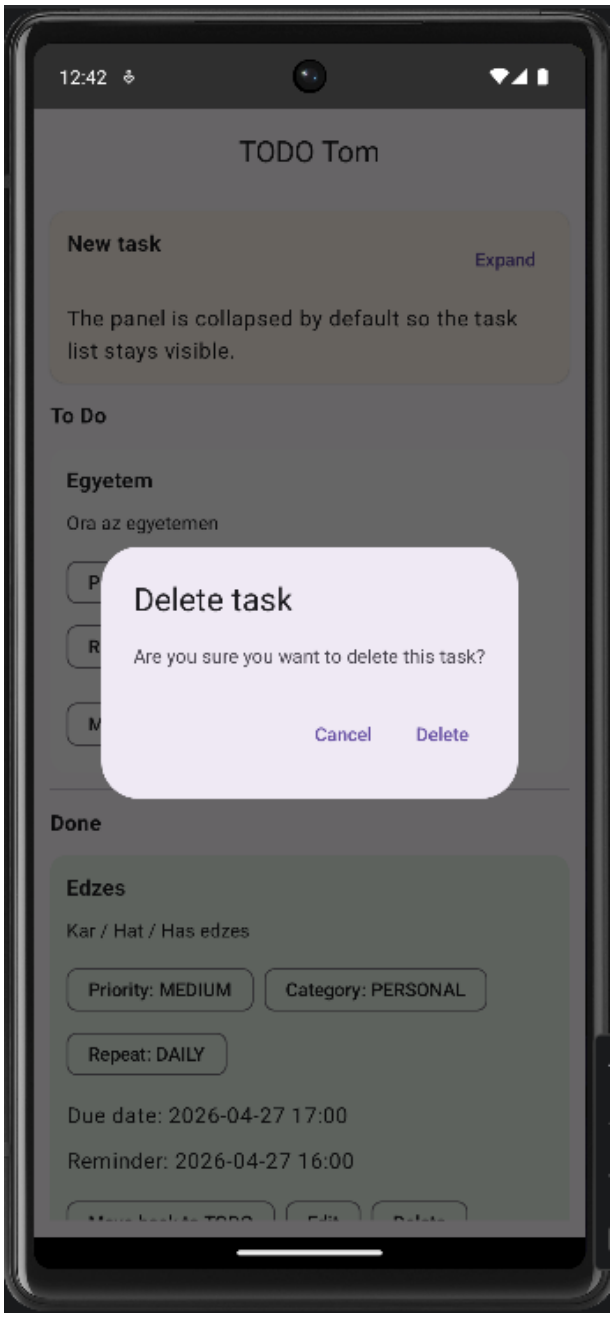
5.4. Task késszerre jelölése / visszaállítása

1. A `To Do` listában a `Mark done` chip felel a Task készre állításáért.
2. A feladat átkerül a `Done` listába.
3. Visszaállítás esetén a `Done` listában a `Move back to TODO` chip felel a feladat visszaállításáért.



6. ábra. A Task szerkesztése

5.5. Task törlése



7. ábra. Task törlése

6. Tesztelés

- Backend CRUD végpontok ellenőrzése
- SQLite perzisztencia és migráció validálása
- Android oldali létrehozás/szerkesztés/törlés/státusz váltás tesztelése
- Emlékeztető folyamat (WorkManager + notification) ellenőrzése

7. Összefoglalás

A TODO Tom egy jól strukturált, gyakorlati alkalmazás, amelyben a backend és front-end felelősségi köre tisztán elválik. A forráskód alapján a rendszer legnagyobb erőssége a gyakorlati használhatóság és a bővíthetőség: az alap CRUD folyamatok mellett haladó funkcióval is rendelkezik.

A következő minőségi lépés a teljes körű automatizált tesztelés bevezetése, valamint az API formális szerződésének (pl. OpenAPI) publikálása lehet.